# Sistema Cajero Automático

## Aplicación de Patrones de Diseño

**Programa: tecnólogo en análisis y desarrollo de software**

**Integrantes:**

**Diego Andrés Gutiérrez Nuñez.**

**Juan David Arboleda Perdomo.**

**Juan Manuel Gómez Suárez.**

**David Santiago Erez Rodriguez**

**servicio nacional de aprendizaje – SENA**
**centro la industria, la empresa y los servicios**
**2026**

## Introducción

El presente documento describe el diseño y análisis de un sistema de simulación de cajero automático desarrollado como aplicación de consola. El objetivo del proyecto es implementar operaciones bancarias básicas aplicando patrones de diseño como Command, Strategy y Observer.

El sistema permite realizar operaciones como consulta de saldo, depósitos, retiros y transferencias, además de manejar validaciones, registro de transacciones y notificaciones del sistema.

**Objetivos:**

**Objectivo general:**

Desarrollar un sistema de cajero automático que permita gestionar operaciones bancarias aplicando patrones de diseño y buenas prácticas de programación.

**Objetivos específicos**

- Implementar operaciones bancarias básicas.
- Aplicar patrones de diseño.
- Diseñar diagramas UML del sistema.
- Validar operaciones financieras.

**Descripción del sistema:**

El sistema simula el funcionamiento de un cajero automático.
Los usuarios pueden interactuar mediante un menú para realizar operaciones sobre sus cuentas bancarias.

**Requerimientos funcionales:**

1. **Consultar saldo:**
   - El usuario puede visualizar el estado actual de su cuenta.
2. **Depositar dinero:**
   - El usuario puede ingresar una cantidad de dinero que será sumada al saldo.
3. **Retirar dinero:**
   - El usuario puede retirar dinero si posee saldo suficiente.
4. **Transferir dinero:**

- El usuario puede transferir dinero a otra cuenta.

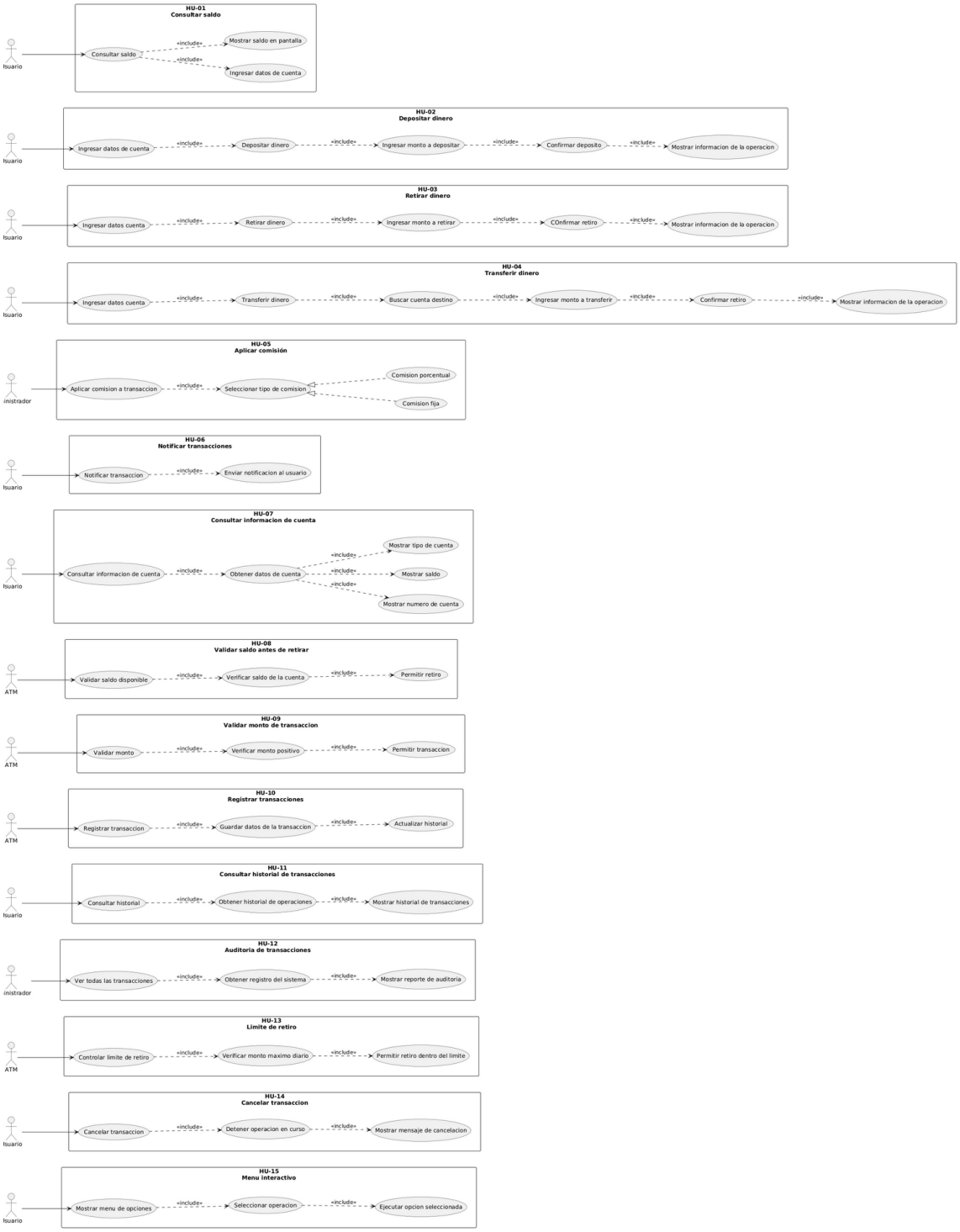
5. **Aplicar comisión:**
   - Cada transacción puede aplicar una comisión dependiendo de la estrategia configurada.
6. **Consultar información de cuenta:**
   - Mostrar número de cuenta, saldo y tipo de cuenta.
7. **Validación de saldo antes de retirar:**
   - El sistema debe impedir retiros si el saldo es insuficiente.
8. **Validación de monto:**
   - No se deben permitir montos negativos ni iguales a cero.
9. **Registro de transacciones:**
   - Cada operación debe quedar registrada en un historial.
10. **Consulta de historial:**
    - El usuario puede ver las operaciones realizadas.
11. **Registro de auditoría:**
    - El sistema debe registrar todas las transacciones realizadas.
12. **Notificaciones del sistema:**
    - Se debe mostrar una alerta cuando se realice una operación importa
13. **Límite de retiro:**
    - El sistema puede limitar el monto máximo de retiro diario.
14. **Cancelación de transacción:**
    - Una transacción puede cancelarse antes de ejecutarse.
15. **Menú interactivo:**
    - El sistema debe mostrar un menú para seleccionar operaciones.
16. **Autenticación de usuario:**
    - El usuario debe ingresar un PIN antes de acceder a su cuenta.
17. **Bloqueo por intentos fallidos:**
    - La cuenta se bloquea después de varios intentos incorrectos.

**Historias de usuario:**

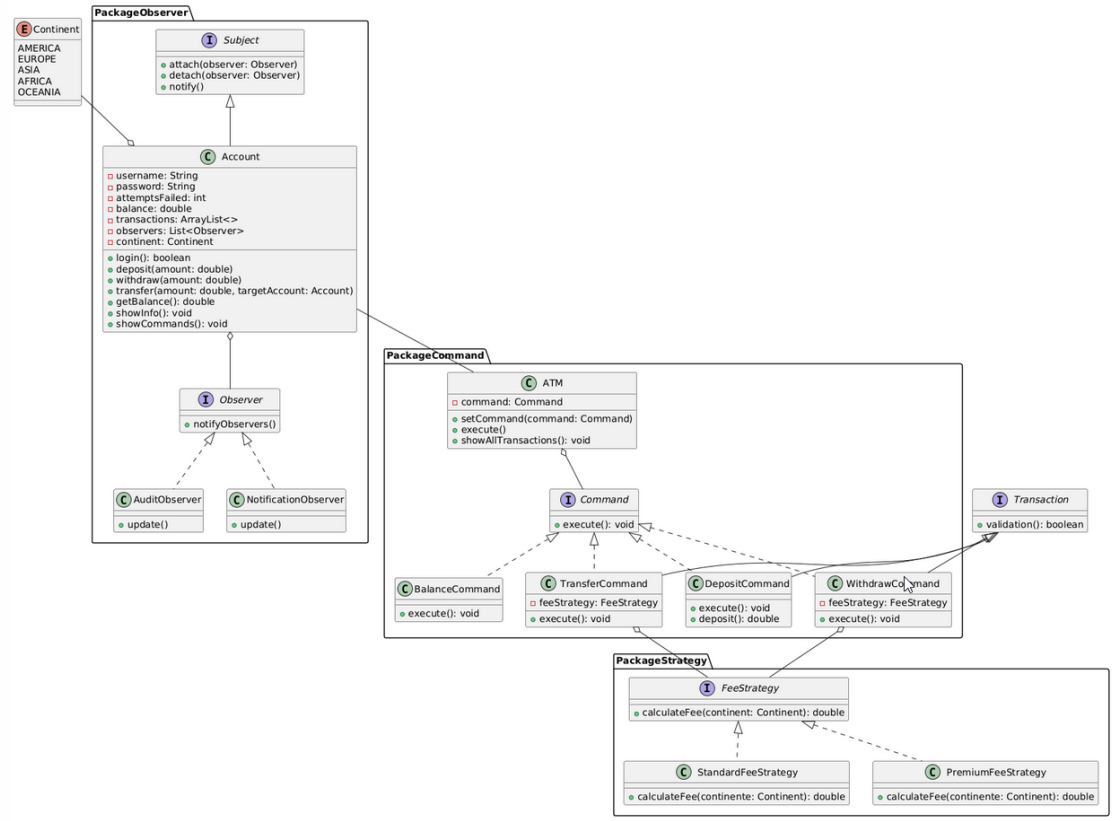
| ID | Historia de Usuario | Explicación Técnica |
|---|---|---|
| **HU-01** | Como usuario del cajero automático quiero consultar el saldo de mi cuenta para saber cuánto dinero tengo disponible. | La consulta de saldo se ejecuta como un comando BalanceCommand cuando el usuario selecciona la opción. |
| **HU-02** | Como usuario del cajero automático quiero depositar dinero en mi cuenta para aumentar mi saldo disponible. | El depósito se ejecuta mediante DepositCommand y el sistema notifica la operación utilizando el patrón Observer. |
| **HU-03** | Como usuario del cajero automático quiero retirar dinero de mi cuenta para tener efectivo disponible. | El retiro se ejecuta mediante WithdrawCommand. Puede aplicar comisión usando FeeStrategy y genera notificación mediante Observer. |
| **HU-04** | Como usuario del cajero automático quiero transferir dinero a otra cuenta para enviar dinero a otra persona. | La transferencia se ejecuta mediante TransferCommand, puede aplicar comisión y genera notificación al sistema. |
| **HU-05** | Como administrador del sistema quiero aplicar diferentes tipos de comisión a las transacciones para gestionar los costos del servicio. | El sistema utiliza estrategias de comisión mediante la interfaz FeeStrategy. |
| **HU-06** | Como usuario quiero recibir notificaciones cuando se realice una transacción para estar informado de los movimientos de mi cuenta. | El sistema notifica a los observadores cuando ocurre una transacción usando el patrón Observer. |
| **HU-07** | Como usuario quiero consultar la información completa de mi cuenta para conocer los datos generales de la misma. | La información se obtiene mediante el método Account.showInfo(). |
| **HU-08** | Como sistema quiero validar que exista saldo suficiente antes de permitir un retiro para evitar operaciones inválidas. | La validación se realiza en WithdrawCommand.validation(). |

| | | |
|---|---|---|
| **HU-09** | Como sistema quiero verificar que el monto de una transacción sea válido antes de procesarla. | La validación se define en Transaction.validation(). |
| **HU-10** | Como sistema quiero registrar todas las transacciones realizadas para mantener un historial de operaciones. | Las transacciones se almacenan en Account.transactions[]. |
| **HU-11** | Como usuario quiero consultar el historial de mis transacciones para revisar mis movimientos anteriores. | El historial se muestra mediante Account.showCommand(). |
| **HU-12** | Como administrador quiero ver todas las transacciones del sistema para poder auditar las operaciones. | El sistema permite visualizar todas las transacciones con ATM.showAllTransactions(). |
| **HU-13** | Como sistema quiero establecer un límite máximo de retiro para evitar retiros excesivos. | El control del límite se gestiona desde la clase Account. |
| **HU-14** | Como usuario quiero cancelar una transacción en curso para detener una operación si cometí un error. | La cancelación se define mediante Transaction.cancel(). |
| **HU-15** | Como usuario quiero interactuar con un menú que me permita elegir las operaciones disponibles. | El menú es gestionado por la clase Main. |
| **HU-16** | Como usuario quiero iniciar sesión con mis credenciales para acceder a mi cuenta. | La autenticación se realiza mediante Account.login(). |
| **HU-17** | Como sistema quiero bloquear una cuenta después de varios intentos fallidos para proteger la seguridad del usuario. | El control de intentos se gestiona con Account.attempts. |

# Diagramas de casos de uso:



## Diagrama de Clases:




## Patrones de diseño utilizados:

### Command

Se utiliza para encapsular las transacciones:

- ConsultarSaldoCommand
- DepositCommand
- WithdrawCommand
- TransferCommand

### Strategy

Se utiliza para manejar las comisiones de transacción.

Ejemplos:

- FixedCommissionStrategy

- PercentageCommissionStrategy

**Observer**

Permite notificar eventos cuando ocurre una transacción.

Ejemplo:

- NotificationObserver
- TransactionLogger

**Conclusiones:**

El desarrollo del sistema permitió aplicar conceptos de diseño orientado a objetos y patrones de diseño para estructurar una aplicación modular y escalable.